

# CC3001 Control 2 Otoño 2025

Profs. Iván Sipirán, Nelson Baloian y Patricio Poblete

27 de junio de 2025

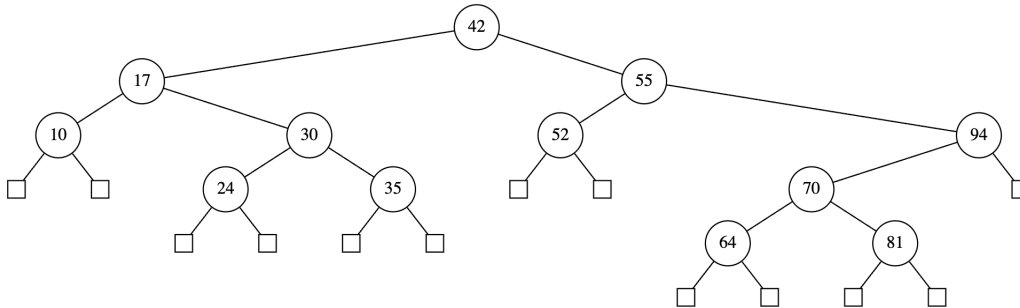
## Instrucciones:

- Tiempo: 2 horas
- El control es INDIVIDUAL
- Entregue cada pregunta en una hoja separada
- No olvide poner su nombre en cada hoja
- Está permitido usar apuntes, ya sea en papel o en formato digital
- Si usa un dispositivo para revisar su apunte en formato digital, está completamente prohibido utilizarlo para cualquier otra finalidad. En particular, no se puede usar chat, ni mail ni ninguna otra forma de comunicación con otra persona o inteligencia artificial. Tampoco puede usarlo para fotografiar el enunciado.

## P1. Nombre: \_\_\_\_\_

Escriba una función  $ancestro(p, x, y)$  que reciba un puntero  $p$  a la raíz de un ABB y dos valores  $x, y$ , con  $x < y$  tales que ambos son llaves almacenadas en nodos del árbol, y que retorne un puntero al ancestro común más cercano entre estos dos nodos.

Por ejemplo, en el siguiente árbol el ancestro común más cercano entre 10 y 35 es el nodo 17, y el ancestro común más cercano entre 55 y 70 es el nodo 55.



Su solución puede ser recursiva o iterativa. Usted puede aprovechar que se sabe que  $x < y$  para que su código resulte más simple.

### No recursivo

```
[ ] def ancestro(p,x,y):  
    while y<p.info or x>p.info:  
        p=p.izq if y<p.info else p.der  
    return p
```

```
[ ] print(ancestro(a.raiz,10,35).info)
```

⇒ 17

```
▶ print(ancestro(a.raiz,55,70).info)
```

⇒ 55

### Recursivo

```
[9] def ancestro(p,x,y):  
    if y<p.info:  
        return ancestro(p.izq,x,y)  
    if x>p.info:  
        return ancestro(p.der,x,y)  
    return p
```

```
[10] print(ancestro(a.raiz,10,35).info)
```

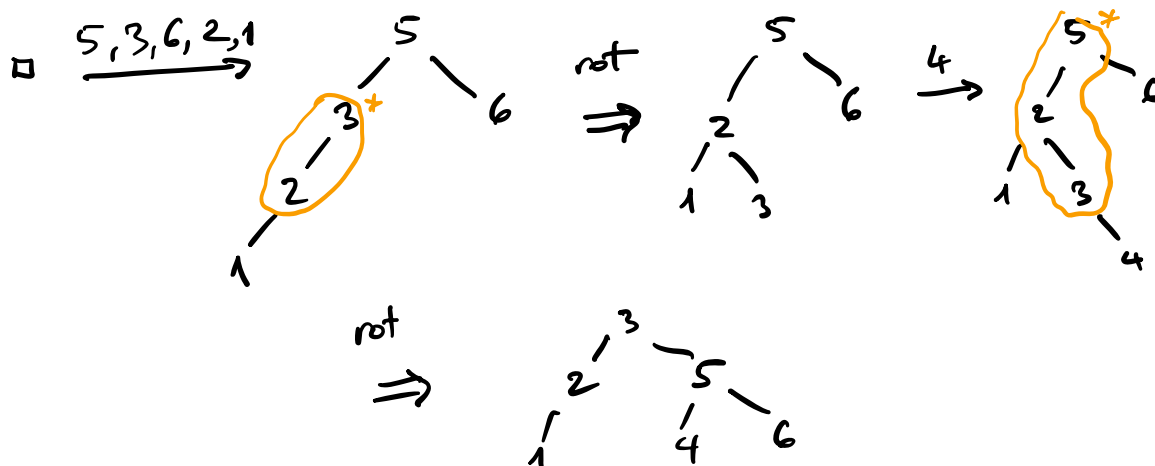
⇒ 17

```
[11] print(ancestro(a.raiz,55,70).info)
```

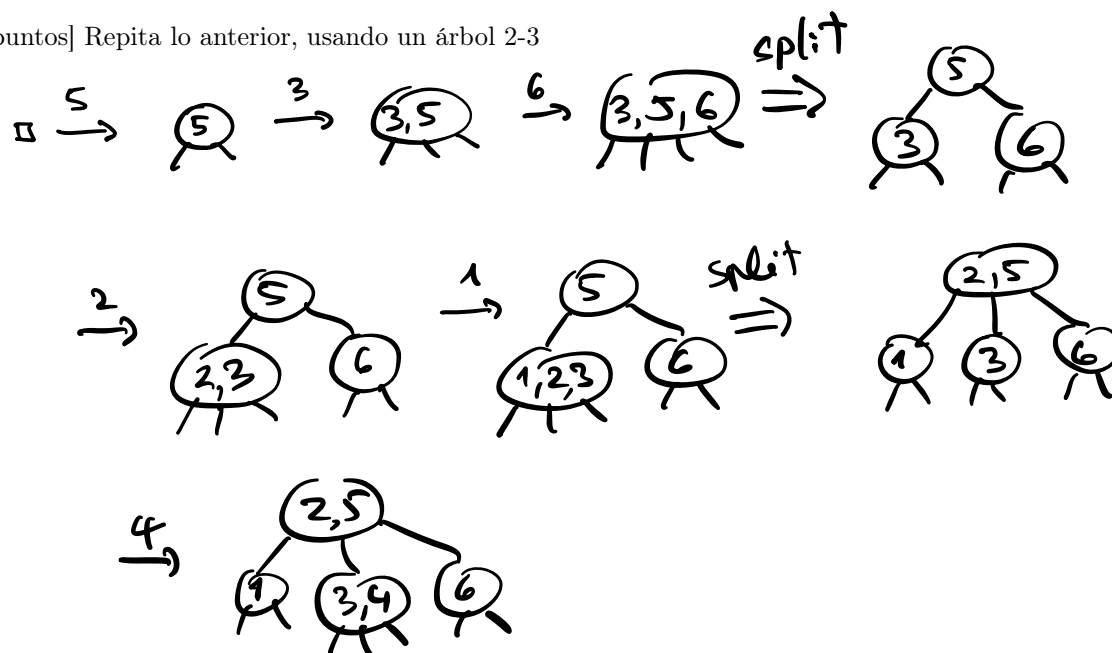
⇒ 55

P2. Nombre: \_\_\_\_\_

1. [2 puntos] Comenzando con un árbol AVL vacío, inserte la secuencia de llaves 5,3,6,2,1,4, indicando cada una de las rotaciones que necesite hacer:



2. [2 puntos] Repita lo anterior, usando un árbol 2-3



3. [2 puntos] Considere una tabla de hashing con Linear Probing con  $m = 10$  y la función de hashing  $h(x) = x \% 10$  y muestre la tabla resultante al insertar la siguiente secuencia de llaves:

34, 89, 20, 49, 26, 15, 74, 30

|    |    |    |   |    |    |    |    |   |    |
|----|----|----|---|----|----|----|----|---|----|
| 20 | 49 | 30 |   | 34 | 15 | 26 | 74 |   | 89 |
| 0  | 1  | 2  | 3 | 4  | 5  | 6  | 7  | 8 | 9  |

Recuerde que en clases se vio el algoritmo Quickselect que, dado un arreglo desordenado  $a[0], \dots, a[n-1]$ , y un entero  $k$  tal que  $0 \leq k \leq n-1$ , coloca en la posición  $a[k]$  el elemento que quedaría ahí si el arreglo estuviera ordenado. El objetivo de esta pregunta es diseñar un algoritmo que realice esto mismo dada una lista de posiciones  $K = [k_0, k_1, \dots]$ , esto es, que coloque en las posiciones  $a[k_0], a[k_1], \dots$  los elementos que quedarían ahí si el arreglo estuviera ordenado. Se puede suponer que los valores de la lista  $K$  están en el rango  $0..n-1$ , que son todos distintos y que están en orden ascendente. Una posibilidad sería llamar a Quickselect múltiples veces, pero lo que se pide es diseñar una sola función que haga todo de una sola vez.

Describa su algoritmo en palabras, no es necesario escribir Python.

Indicación: Como caso de borde, si la lista  $K$  es vacía, el algoritmo no debe hacer nada.

Definimos una función auxiliar

Quickselect( $a, i, j, K$ )

que opera solo sobre el rango  $a[i], \dots, a[j]$

(La llamada inicial sería Quickselect( $a, 0, \text{len}(a)-1, K$ ))

La función hará lo siguiente:

- 1) Hacer una partición de Quicksort con un elemento aleatorio, obteniendo en la variable  $p$  la posición en la que queda ese elemento

$p = \text{partición}(a, i, j)$

- 2) Particionar  $K$  en 3 sub listas

$K_{\text{izq}} = [k \in K \text{ tal que } k < p]$

$K_{\text{exacto}} = [k \in K \text{ tal que } k = p]$

$K_{\text{der}} = [k \in K \text{ tal que } k > p]$

- 3) Hacer Quickselect recursivo a ambos lados:

Quickselect( $a, i, p-1, K_{\text{izq}}$ )

Quickselect( $a, p+1, j, K_{\text{der}}$ )

Nótese que Quickselect( $a, i, j, []$ ) no hace nada

Nótese también que  $K_{\text{exacto}}$  puede ser vacía o contener solo 1 elemento.

En el apunte aparece el siguiente método `merge`, que mezcla dos listas ordenadas `a` y `b`:

```
def merge(a,b):
    i=0
    j=0
    while i<len(a) or j<len(b):
        if j>=len(b) or (i<len(a) and a[i]<=b[j]):
            yield a[i]
            i=i+1
        else:
            yield b[j]
            j=j+1
```

En esta pregunta se le pide suponer que cada una de las listas `a` y `b` no contiene elementos duplicados, pero sí puede haber elementos que estén en `a` y en `b`, y usted debe modificar la función `merge` para que el resultado producido no contenga duplicados, esto es, que cada elemento que esté en `a` y en `b` aparezca solo una vez en la salida.

Por ejemplo, si `a=[10, 24, 56, 75, 89]` y `b=[10, 34, 45, 75, 96]`, la salida debe ser 10, 24, 34, 45, 56, 75, 89, 96.

```
[1] def merge(a,b):
    i=0
    j=0
    while i<len(a) or j<len(b):
        if j>=len(b):
            yield a[i]
            i=i+1
        elif i>=len(a):
            yield b[j]
            j=j+1
        elif a[i]<b[j]:
            yield a[i]
            i=i+1
        elif a[i]>b[j]:
            yield b[j]
            j=j+1
        else:
            yield a[i]
            i=i+1
            j=j+1
```

```
[2] a=[10, 24, 56, 75, 89]
    b=[10, 34, 45, 75, 96]
    print(a)
    print(b)
    c=[x for x in merge(a,b)]
    print(c)
```

```
⇒ [10, 24, 56, 75, 89]
   [10, 34, 45, 75, 96]
   [10, 24, 34, 45, 56, 75, 89, 96]
```